

```
pi@raspberrypi:~$ dmesg
pi@raspberrypi:~$ sudo insmod gpio_driver.ko
pi@raspberrypi:~$ dmesg
[ 1589.261117] Major number allocated to BCM GPIO 247
[ 1589.261140] Registering GPIO Platform Driver
[ 1589.261347] Registering GPIO Platform Driver
[ 1589.261442] BCM GPIO Probe called...
pi@raspberrypi:~$
```

Figure 3: *dmesg* output

- Run '*sudo dmesg -C*'. (This command would clean up all the kernel boot print logs.)
- Run '*sudo make*'. (This command would compile GPIO driver. Do this only for the second method.)
- Run '*sudo insmod gpio_driver.ko*'. (This command inserts the driver into the OS.)
- Run '*dmesg*'. You can see the prints from the GPIO driver and the major number allocated to it, as shown in Figure 3. (The major number plays a unique role in identifying a specific driver with whom the process from the application space wants to communicate with, whereas the minor number is used to recognise hardware.)
- Run '*sudo mknod /dev/bcm-gpio c major-num 0*'. (The '*mknod*' command creates a node in */dev* directory, '*c*' stands for character device and '*0*' is the minor number.)
- Run '*sudo gcc gpio_app.c -o gpio_app*'. (Compile the GPIO control application.)

Now let's test our GPIO driver and application. To verify whether our driver is indeed communicating with GPIO, short pins 25 and 24 (one can use other available pins like 17, 22 and 23 as well but make sure that they aren't mixed up for any other peripheral) using the female-to-female jumper (Figure 4). The default values of both the pins will be 0. To confirm the default values, run the following commands:

```
sudo ./app -n 25 -g 1
```

This will be the output. The output value of GPIO 25 = 0. Now run the following command:

```
sudo ./app -n 24 -g 1
```

This will again be the output. The output value of GPIO 24 = 0.

That's it. It's verified (see Figure 5). Now, as the GPIO pins are shorted, if we output 1 to 24 then it would be the input value of 25 and vice versa.

To test this, run:

```
sudo ./app -n 24 -d 1 -v 1 -s 1
```



Figure 4: R-pi GPIO

```
pi@raspberrypi:~$ sudo mknod /dev/bcm-gpio c 247 0
pi@raspberrypi:~$ ls -al /dev/bcm-gpio
lrwxrwxrwx 1 root root 247  0 Jul 20 08:39 /dev/bcm-gpio
pi@raspberrypi:~$ sudo ./gpio_app -n 24 -g 1
GPIO number : 24
GPIO direction : 0
GPIO value : 0
The output value of GPIO 24 = 0
pi@raspberrypi:~$ ./gpio_app -n 25 -g 1
GPIO number : 25
GPIO direction : 0
GPIO value : 0
The output value of GPIO 25 = 0
pi@raspberrypi:~$
```

Figure 5: Output showing GPIO 24 = 0

```
pi@raspberrypi:~$ sudo ./gpio_app -n 24 -d 1 -v 1 -s 1
GPIO number : 24
GPIO direction : 1
GPIO value : 1
gpio set direction
gpio set value
The GPIO 24 is set with value 1 (Direction 1)
pi@raspberrypi:~$ ./gpio_app -n 25 -g 1
GPIO number : 25
GPIO direction : 0
GPIO value : 0
The output value of GPIO 25 = 1
pi@raspberrypi:~$ sudo ./gpio_app -n 24 -g 1
GPIO number : 24
GPIO direction : 0
GPIO value : 0
The output value of GPIO 24 = 1
pi@raspberrypi:~$
```

Figure 6: Output showing GPIO 25 = 1

This command will drive the value of GPIO 24 to 1, which in turn will be routed to GPIO 25. To verify the value of GPIO 25, run:

```
sudo ./app -n 25 -g 1
```

This will give the output. The output value of GPIO 25 = 1 (see Figure 6).

One can also connect any external device or a simple LED (through a resistor) to the GPIO pin and test its output.

Arguments passed to the application through the command lines are:

```
-n : GPIO number
-d : GPIO direction (0 - IN or 1 - OUT)
-v : GPIO value (0 or 1)
-s/g: set/get GPIO
```

The files are:

```
gpio_driver.c : GPIO driver file
gpio_app.c : GPIO control application
gpio.h : GPIO header file
Makefile : File to compile GPIO driver
```

After conducting this experiment, some curious folk may have questions like:

- Why does one have to use virtual addresses to access GPIO?
- How does one determine the virtual address from the physical address?

We will discuss the answers to these in later articles. **END** 

By: Sumeet Jain

The author works at *elnfochips* as an embedded systems engineer. You can reach him at sumeet.jain@elnfochips.com